DEEPDIVE

Requirements, Architecture & Phased Implementation Plan

Hackathon project blueprint for a multi-agent research system that verifies claims against their cited sources.

Project goal	Produce structured, cited research reports where each factual claim is checked against retrieved source evidence.
Implementation style	API-first backend, plain Python functions and loops, team-based delivery.
Core principle	A citation is not enough: the cited source passage must actually support the claim.

Prepared as a practical build plan

This document consolidates the challenge requirements, proposed architecture, API-first approach, repository layout, implementation milestones, and evaluation plan. It is designed for a team new to LLM application development.

Recommended approach: prove one end-to-end path first, then add research, claim-level verification, bounded retries, audit logging, and evaluation.

1. Executive Summary

DeepDive is not intended to be a basic search wrapper. It is a research workflow that gathers evidence, drafts claims, checks each claim against the source it cites, and repeats research when important areas remain weakly supported. It must preserve an audit trail so a human can inspect what was accepted, rejected, or left unresolved.

The key engineering challenge is citation correctness: whether a cited source actually supports the statement attached to it. The system should prefer an explicit uncertainty or evidence gap over an unsupported claim.

What the first release should demonstrate

- A user can submit a research question through an HTTP API.
- A planner creates sub-questions that define the research scope.
- Researchers retrieve real sources and preserve the relevant passages.
- A draft is represented as individual claims linked to evidence IDs.
- A verifier labels each claim as supported, contradicted, or insufficiently supported.
- A bounded loop sends weak areas back for additional research.
- The final report and audit history can be inspected.
- A repeatable test set measures grounding and coverage.

2. Challenge Requirements

Requirement	What it means in DeepDive	Evidence of completion
Multi-agent orchestration	At least three agents with distinct responsibilities, instructions, or tools. A coordinator passes outputs between them.	Planner, Researcher, Verifier and preferably Synthesizer operate as distinct logical components.
Real information gathering	Retrieve current web evidence and optionally read user-uploaded documents.	Stored source URL/title, retrieval timestamp and relevant source passage.
Claim-level verification	Check each factual claim against the cited passage, not just whether a URL exists.	Structured verdict and reason for each claim.
Research feedback loop	If evidence is thin or a claim is unsupported, perform another targeted research pass.	Retry counter, stopping condition, and a recorded loop decision.
Grounded cited report	Final factual claims must trace to retrieved evidence. Unsupported claims are removed or clearly flagged.	Claim-to-evidence mapping preserved through synthesis.
Audit trail	Record sub-questions, searches, evidence, decisions, rejections, and reasons.	Run history can be inspected after completion.
Self-evaluation	Test the system against a defined set of questions and assess citation correctness and completeness.	Evaluation script or report with metrics and recorded results.

Optional bonus capabilities

- Use LangGraph for explicit workflow nodes and conditional loops (not required for the initial version).
- Detect contradictions across sources and surface disagreement rather than silently choosing one.
- Weight source quality using visible, explainable rules; a primary source is not automatically authoritative for every question.
- Export reports to PDF or a shareable page, and support follow-up questions using prior evidence.

3. Scope and Design Principles

Build the smallest version that proves the trust mechanism. Avoid adding advanced infrastructure before the core claim-to-evidence flow works.

In scope for the first usable release

- API-first interaction; one research run per request initially.
- Mesh API for every LLM call, as required by the hackathon.
- A web search/retrieval provider separate from the LLM gateway.
- Structured inputs and outputs for agent handoffs.
- Persistent evidence, claims, verdicts, and audit events.
- Bounded research retries and a clear terminal state for unresolved gaps.
- A small, manually reviewed evaluation set.

Defer until the core workflow is reliable

- Vector database, unless large document collections or semantic retrieval make it necessary.
- Uploaded-document support, PDF export, and conversational follow-ups.
- Complex autonomous agent frameworks or separate microservices.

Reliability rules

- Treat retrieved web content as untrusted data, not as instructions to the agents.
- A model-generated verdict is a useful check, not proof of truth; measure verifier errors.
- Never invent source URLs, titles, quotations, publication dates, or evidence.
- If a source cannot be retrieved or the passage does not support the claim, mark the claim unsupported or unresolved.
- Use a maximum retry count and stop when the evidence gap cannot be resolved.
- Re-verify factual claims introduced or materially changed during final synthesis.

4. Proposed Architecture

Layer	Component	Responsibility
API	FastAPI	Validates requests, exposes endpoints, returns results and errors.
Orchestration	Python coordinator	Runs planning, research, drafting, verification, retry logic and synthesis.
Agent layer	Planner	Converts the question into sub-questions and research goals.
Agent layer	Researcher	Uses the search/retrieval tools, extracts evidence, stores source metadata.
Agent layer	Verifier	Checks claim-to-passage support and returns verdicts with explanations.
Agent layer	Synthesizer	Writes a structured report using verified claims and explicit limitations.
Integration	Mesh API client	Central wrapper for every LLM call; reads credentials from environment variables.
Retrieval	Search provider	Finds sources and retrieves accessible page content/passages.
Persistence	SQLite initially	Stores runs, sub-questions, evidence, claims, verdicts and audit events.
Testing	Evaluation suite	Measures citation support, unsupported-claim leakage and coverage.

Logical workflow

1. User question	2. Planner	3. Research
Question received by API	Sub-questions and source goals	Retrieve sources and save passages
4. Draft claims	**5. Verify**	**6. Decide**
Each claim links to evidence IDs	Check claim against its cited passage	Keep, reject/flag, or research again
7. Retry if needed	**8. Synthesize**	**9. Audit and return**
Bounded loop for weak coverage	Final report from verified claims	Save decisions and respond to client

5. Data Contracts and Persistence

Define the data shapes before parallel development. Predictable contracts make agent handoffs testable and allow different teammates to work on separate components.

Suggested core records

Record	Important fields	Purpose
ResearchRun	run_id, question, status, started_at, completed_at, retry_count	Represents one user research request.
SubQuestion	sub_question_id, run_id, question, coverage_status	Tracks the planner's research scope and whether it is adequately covered.
Evidence	evidence_id, sub_question_id, source_url, source_title, retrieved_at, passage, source_type	Stores retrievable evidence and provenance.
Claim	claim_id, run_id, claim_text, evidence_ids, status	Represents one independently verifiable factual statement.
Verification	claim_id, verdict, rationale, checked_at, verifier_version	Records why a claim was supported, contradicted, or unresolved.
AuditEvent	event_id, run_id, event_type, details, timestamp	Records searches, decisions, retries, rejections, and finalization.

Example claim object

```
{
  "claim_id": "C-001",
  "claim_text": "A raw-material price change can affect production costs.",
  "evidence_ids": ["E-014"],
  "verification_status": "supported",
  "verification_reason": "The retrieved passage directly describes the cost impact."
}
```

Use stable evidence IDs rather than copying source text everywhere. The final report should preserve the mapping from each claim to its evidence. The cited passage should be retained for verification and auditing, subject to the source's access and usage terms.

6. Repository Structure

```
deepdive/
■■■ app/
■   ■■■ main.py                 # FastAPI routes
■   ■■■ config.py               # Environment configuration
■   ■■■ services/
■   ■   ■■■ llm_client.py       # Mesh API wrapper
■   ■■■ agents/
■   ■   ■■■ planner.py
■   ■   ■■■ researcher.py
■   ■   ■■■ verifier.py
■   ■   ■■■ synthesizer.py
■   ■■■ workflow/
■   ■   ■■■ coordinator.py      # Python orchestration and retry loop
■   ■■■ database/
■   ■   ■■■ repository.py       # Persistence operations
■   ■■■ schemas/
■       ■■■ research.py         # Request/response and agent schemas
■■■ tests/
■   ■■■ test_health.py
■   ■■■ test_verifier.py
■   ■■■ test_workflow.py
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
```

Only the first milestone files need implementation initially. Create the other modules as their phase begins.

7. Proposed API Contract

Endpoint	Initial purpose	Example response shape
GET /health	Confirms the backend is running.	{"status": "ok"}
POST /api/v1/ask	Milestone 1: proves Mesh connectivity.	{"answer": "..."}
POST /api/v1/research	Full workflow in later phases.	{"run_id": "...", "status": "completed", "report": {...}}
GET /api/v1/runs/{run_id}	Later: retrieve report and audit history.	{"run_id": "...", "events": [...], "claims": [...]}

The initial `/api/v1/ask` endpoint is only a connectivity test. It is not the finished DeepDive research endpoint. Keep the initial endpoint small and add `/research` after the planner and researcher are implemented.

8. Phased Implementation Plan

Each phase should have a demonstrable outcome and acceptance criteria. The order prioritizes early integration risk and the core trust requirement.

Phase	Build	Acceptance criteria
0. Team setup	Repository, branches, README, environment policy, coding conventions and API/data contracts.	All contributors can run the app; secrets are excluded from Git; work can be merged through PRs.
1. Mesh connectivity	FastAPI, config loader, Mesh client, GET /health and POST /api/v1/ask.	Health check returns 200; a prompt receives a real Mesh model response.
2. Planner	Planner prompt and validated structured output for 3–5 sub-questions.	Output conforms to schema; invalid output is handled safely.
3. Research retrieval	Integrate a search provider; retrieve source pages/passages; assign evidence IDs; persist results.	Evidence has URL/title/passage/time; failed retrievals are logged; no invented sources.
4. Claims and draft	Create individual claims, each mapped to one or more evidence IDs.	Every draft claim has a traceable evidence mapping or is explicitly marked as needing evidence.
5. Verifier	Check claim against exact cited passages; return supported/contradicted/insufficient_evidence and rationale.	Test cases include supported, contradicted, and unrelated passages; verdicts are structured.
6. Coordinator and loop	Plain Python functions and bounded retries; re-research weak sub-questions; stop on success or limit.	Loop retries only where needed, never runs indefinitely, and unresolved gaps are disclosed.
7. Synthesis and audit	Final report from verified claims; persist searches, evidence, verdicts, rejections, and retry events.	A run can be reconstructed; final factual claims remain linked to verified evidence.
8. Evaluation	Create test questions and labeled claim/evidence examples; calculate metrics; document limitations.	Results are recorded in the repository and failures lead to improvements.
9. Demo polish / bonus	UI, report export, contradiction detection, source weighting, follow-up support.	Core workflow remains reliable; bonus features are demonstrated separately.

9. Milestone 1: Exact Build Steps

Goal: prove that a request can pass through FastAPI, call Mesh, and return a response.

- Create the repository and Python virtual environment.
- Install `fastapi`, `uvicorn[standard]`, `openai`, and `python-dotenv`.
- Create `.env` with `MESH_API_KEY` and `MESH_MODEL`; never commit `.env`.
- Implement `get_required_setting()` in `app/config.py`.
- Implement `ask_model()` in `app/services/llm_client.py` using the OpenAI SDK with Mesh's base URL `https://api.meshapi.ai/v1`.
- Implement `GET /health` and `POST /api/v1/ask` in `app/main.py`.
- Run `python -m uvicorn app.main:app --reload` and test using `/docs`.

Example environment configuration

```
MESH_API_KEY=rsk_your_actual_key_here
MESH_MODEL=your_available_model_id
```

Example local commands (Windows PowerShell)

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload
```

Milestone 1 is complete when `/health` returns HTTP 200, `/api/v1/ask` returns a real model response, configuration errors are clear, and no API key is present in source control.

10. Agent Responsibilities and Retry Logic

Planner

- Input: original user question.
- Output: sub-questions, scope boundaries, and recommended source types.
- Should not claim that research facts are already established.

Researcher

- Input: one or more sub-questions and search constraints.
- Output: source records and exact relevant passages, with provenance.
- Avoids treating snippets as sufficient when the full page is needed; records inaccessible or low-quality sources.

Verifier

- Input: one claim plus the evidence passages cited for it.
- Output: `supported`, `contradicted`, or `insufficient_evidence`, plus a concise rationale and evidence IDs.
- Must not infer that a source supports a claim merely because it discusses the same topic.

Synthesizer

- Input: verified claims, citations, disagreements, and unresolved gaps.
- Output: a structured report whose factual claims remain traceable to verified evidence.
- Any new or materially rewritten factual claim must go through verification again.

Plain Python coordinator logic

```
plan = planner(question)
evidence = research(plan.sub_questions)
claims = draft_claims(question, evidence)

for attempt in range(MAX_RESEARCH_ATTEMPTS + 1):
    verdicts = verify(claims, evidence)
    weak_areas = find_unsupported_claims_and_thin_topics(
        plan, claims, evidence, verdicts
    )

    if not weak_areas:
        break

    if attempt == MAX_RESEARCH_ATTEMPTS:
        break

    new_evidence = research(weak_areas)
    evidence = merge_evidence(evidence, new_evidence)
    claims = update_claims(claims, evidence, weak_areas)

report = synthesize(verified_claims(claims, verdicts), weak_areas)
save_audit_trail(plan, evidence, claims, verdicts, report)
```

This is conceptual pseudocode. Production code must also handle tool failures, duplicate sources, claim IDs, empty evidence, retry budgets, and verification of the final synthesis.

11. Evaluation Plan

Metric	Definition	How to measure
Citation support rate	Share of evaluated cited claims whose cited passages actually support them.	Manually label a sample or a test set; compare to final citations.
Unsupported-claim leakage	Share of unsupported claims that remain in the final report as factual claims.	Seed unsupported draft claims and inspect what survives.
Evidence coverage	Share of planned sub-questions with adequate evidence.	Define a coverage rubric and label each sub-question.
Verifier precision	Of claims the verifier marks supported, how many are truly supported?	Compare verifier verdicts to human labels.
Loop effectiveness	Whether a retry resolves an evidence gap or improves coverage.	Compare before/after coverage and verdicts per retry.
Operational metrics	Latency, source retrieval failures, LLM errors and cost if available.	Record timings and failure counts per run.

Minimum evaluation practice

- Prepare a small test set of diverse research questions.
- Include intentionally unsupported claims and irrelevant cited passages to test rejection behavior.
- Include conflicting sources and time-sensitive claims.
- Keep the expected answer/evidence labels independent of the system being evaluated.
- Record the date, test cases, metrics, and known limitations in the repository.

12. Team Collaboration Plan

Agree on interfaces first. Team members can then implement separate modules in parallel without changing each other's assumptions.

Workstream	Ownership	Deliverables
Backend / Mesh integration	API and integration owner	FastAPI routes, configuration, Mesh wrapper, error handling.
Research retrieval	Research owner	Search integration, source extraction, evidence schema, retrieval tests.
Verification / evaluation	Trust and quality owner	Verifier prompts/schema, test cases, metrics, verifier error analysis.
Workflow / persistence	Orchestration owner	Coordinator, retry logic, SQLite repository, audit events.
Demo / integration	Integration lead or shared role	End-to-end test, report presentation, optional UI/export, demo script.

Suggested Git workflow

- Protect `main`; use short-lived feature branches and pull requests.
- Require at least one teammate review for core workflow changes.
- Keep `.env` and credentials out of Git; commit `.env.example` instead.
- Add tests for schemas and workflow branches, especially retry-limit and unsupported-claim cases.
- Agree on the shape of agent outputs and database records before parallel implementation.

13. Risks and Mitigations

Risk	Mitigation
Verifier incorrectly approves an unsupported claim	Evaluate against human-labeled cases; require exact passage-level evidence; report uncertainty.
Search returns snippets but no accessible source content	Retrieve the source page where permitted; otherwise label evidence limitations and avoid strong claims.
Loop runs indefinitely or repeats the same searches	Set retry/search budgets, deduplicate sources and queries, and define stopping conditions.
Model output is malformed	Use Pydantic validation, structured output where supported, and bounded correction attempts.
Sources disagree	Preserve both evidence records and flag the contradiction with dates and scope.
API key is exposed	Environment variables/secrets manager, `.gitignore`, key rotation if leaked.
Final synthesis adds new unsupported facts	Re-extract or track final claims and run a final verification pass.

14. Definition of Done

- A user can submit a question and receive a structured research report through an API.
- The system retrieves real sources and stores traceable evidence.

- Each final factual claim is linked to source evidence and has a verification status.
- Unsupported claims are removed or explicitly flagged; gaps trigger bounded additional research.
- The audit trail explains searches, accepted/rejected claims, retries, and reasons.
- A test set and evaluation results are committed to the repository.
- The demo can show at least one supported claim and one rejected or unresolved claim.

Recommended next action

Start with Phase 0 and Milestone 1 only. Get the FastAPI-to-Mesh call working, commit a clean project skeleton, and then build the Planner. Add the researcher and verifier after the API integration is stable.

Implementation note: Mesh model IDs, API access, search-provider selection, and any rate limits must be confirmed against the accounts and documentation available to the team. The plan deliberately avoids hard-coding a specific model or search provider.